Q1. Write a Python program to manage the borrowing records of books in a library. Implement the following functionalities: • Compute the average number of books borrowed by all library members. • Find the book with the highest and lowest number of borrowings in the library. • Count the number of members who have not borrowed any books (denoted by a borrow count of 0). • Display the most frequently borrowed book (i.e., the mode of borrow counts). After performing, determine the time and Space complexity of each operation

```
#GA-1: Library borrowing records - 3 dictionaries (easy version)
# members : member name -> books borrowed
# books : book name -> times borrowed
# freq : borrow count -> number of members with that count

members = {}
books = {}
freq = {}

n = int(input("Number of members: "))
for i in range(n):
name = input("Member name: ")
members[name] = int(input("Books borrowed: "))

m = int(input("Number of books: "))
for i in range(m):
name = input("Book name: ")
books[name] = int(input("Times borrowed: "))

# 1. Average - Time O(n), Space O(1)
total = 0
for name in members:
total = total + members[name]
print("Average borrowed:", total / n)

# 2. Highest and lowest book - Time O(m), Space O(1)
highest = max(books, key=books.get)
lowest = min(books, key=books.get)
print("Highest borrowed book:", highest, books[highest])
print("Lowest borrowed book:", lowest, books[lowest])

# 3. Members with 0 borrows - Time O(n), Space O(1)
zero = 0
for name in members:
if members[name] == 0:
zero = zero + 1
print("Members with 0 borrows:", zero)

# 4. Mode - Time O(n), Space O(k)
for name in members:
c = members[name]
freq[c] = freq.get(c, 0) + 1

mode = max(freq, key=freq.get)
print("Mode of borrow counts:", mode, "appears", freq[mode], "times")
```

Q 4 Implementing a real-time undo/redo system for a text editing application using a Stack data structure. The system should support the following operations: - Make a Change: A new change to the document is made. - Undo Action: Revert the most recent change and store it for potential redo. - Redo Action: Reapply the most recently undone action. - Display Document State: Show the current state of the document after undoing or redoing an action

```
# GB-4: Undo/Redo system for a text editor using Stacks (simple version)


undo_stack = []   # stores changes that are currently applied
redo_stack = []   # stores changes that were undone



def make_change(text):
    undo_stack.append(text)
    redo_stack.clear()         # a new change invalidates the redo history
    print("Change made:", text)



def undo():
    if len(undo_stack) == 0:
        print("Nothing to undo")
    else:
        last = undo_stack.pop()
        redo_stack.append(last)
        print("Undone:", last)



def redo():
    if len(redo_stack) == 0:
        print("Nothing to redo")
    else:
        last = redo_stack.pop()
```

```
        undo_stack.append(last)
        print("Redone:", last)


def display():
    if len(undo_stack) == 0:
        print("Document is empty")
    else:
        print("Document:", " ".join(undo_stack))


while True:
    print("\n----- MENU -----")
    print("1. Make a change")
    print("2. Undo")
    print("3. Redo")
    print("4. Display document")
    print("5. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        text = input("Enter text to add: ")
        make_change(text)
    elif choice == "2":
        undo()
    elif choice == "3":
        redo()
    elif choice == "4":
        display()
    elif choice == "5":
        print("Exiting...")
```

```
        break
    else:
        print("Invalid choice")
```

Q 7 Implement a hash table of size 10 and use the division method as a hash function. In case of a collision, use chaining. Implement the following operations: • Insert(key): Insert key-value pairs into the hash table. • Search(key): Search for the value associated with a given key. • Delete(key): Delete a key-value pair from the hash table

take input from user use froper function and use directory if required

```
# GC-7: Hash table of size 10, division method, collision handling by chaining

SIZE = 10
table = [[] for _ in range(SIZE)]   # each slot holds a chain (list) of [key, value] pairs


def hash_function(key):
    """Division method"""
    return key % SIZE


def insert(key, value):
    index = hash_function(key)
    for pair in table[index]:
        if pair[0] == key:            # key already exists -> update value
            pair[1] = value
            print("Key", key, "already exists. Value updated.")
            return
    table[index].append([key, value])   # collision? just add to the chain
    print("Inserted (", key, ",", value, ") at index", index)
```

```
def search(key):
    index = hash_function(key)
    for pair in table[index]:
        if pair[0] == key:
            print("Found: key =", key, ", value =", pair[1], ", index =", index)
            return pair[1]
    print("Key", key, "not found")
    return None


def delete(key):
    index = hash_function(key)
    for pair in table[index]:
        if pair[0] == key:
            table[index].remove(pair)
            print("Deleted key", key, "from index", index)
            return
    print("Key", key, "not found. Nothing deleted")


def display():
    print("\nIndex | Chain")
    for i in range(SIZE):
        if len(table[i]) == 0:
            print(i, "   | empty")
        else:
            chain = ""
            for pair in table[i]:
                chain = chain + "(" + str(pair[0]) + ":" + str(pair[1]) + ") -> "
            print(i, "   |", chain + "None")
```

```
def read_key():
    while True:
        try:
            return int(input("Enter key (integer): "))
        except ValueError:
            print("Invalid key. Enter an integer.")



while True:
    print("\n----- HASH TABLE (Chaining) -----")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        k = read_key()
        v = input("Enter value: ")
        insert(k, v)
    elif choice == "2":
        k = read_key()
        search(k)
    elif choice == "3":
        k = read_key()
        delete(k)
    elif choice == "4":
        display()
    elif choice == "5":
```

```
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

Q 10

```
# GD-10: Binary Search Tree using dictionaries as nodes
# Each node is a dictionary: {"data": value, "left": None, "right": None}


def create_node(data):
    return {"data": data, "left": None, "right": None}


def insert(root, data):
    if root is None:
        return create_node(data)
    if data < root["data"]:
        root["left"] = insert(root["left"], data)
    elif data > root["data"]:
        root["right"] = insert(root["right"], data)
    else:
        print("Duplicate value, not inserted")
    return root


def search(root, key):
    if root is None:
        return False
    if key == root["data"]:
        return True
```

```
    elif key < root["data"]:
        return search(root["left"], key)
    else:
        return search(root["right"], key)


def find_min(root):
    while root["left"] is not None:
        root = root["left"]
    return root


def delete(root, key):
    if root is None:
        print("Key not found")
        return None

    if key < root["data"]:
        root["left"] = delete(root["left"], key)
    elif key > root["data"]:
        root["right"] = delete(root["right"], key)
    else:
        # Case 1: no child (leaf)
        if root["left"] is None and root["right"] is None:
            return None
        # Case 2: one child
        if root["left"] is None:
            return root["right"]
        if root["right"] is None:
            return root["left"]
        # Case 3: two children -> replace with inorder successor
```

```
        successor = find_min(root["right"])
        root["data"] = successor["data"]
        root["right"] = delete(root["right"], successor["data"])
    return root


def inorder(root):
    if root is not None:
        inorder(root["left"])
        print(root["data"], end=" ")
        inorder(root["right"])


def preorder(root):
    if root is not None:
        print(root["data"], end=" ")
        preorder(root["left"])
        preorder(root["right"])


def postorder(root):
    if root is not None:
        postorder(root["left"])
        postorder(root["right"])
        print(root["data"], end=" ")


def display(root):
    if root is None:
        print("Tree is empty")
        return
```

```
    print("Inorder   :", end=" ")
    inorder(root)
    print("\nPreorder  :", end=" ")
    preorder(root)
    print("\nPostorder :", end=" ")
    postorder(root)
    print()


root = None

while True:
    print("\n----- BINARY SEARCH TREE -----")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter choice: "))

    if choice == 1:
        n = int(input("Enter value to insert: "))
        root = insert(root, n)
    elif choice == 2:
        n = int(input("Enter value to delete: "))
        root = delete(root, n)
    elif choice == 3:
        n = int(input("Enter value to search: "))
        if search(root, n):
            print(n, "is FOUND in the tree")
        else:
```

```
        print(n, "is NOT found in the tree")
    elif choice == 4:
        display(root)
    elif choice == 5:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

Q3 Problem statement: In a company, employee salaries are stored in a list as floating-point numbers. Write a python program that sorts the employee salaries in ascending order using following two algorithms: 1. Selection sort: Sort the values using the selection sort algorithm 2. Bubble sort: Sort the salaries using bubble sort algorithm After sorting the salaries, the program should display top 5 highest salaries in the company.

takeinput from user use functions and use dictnoru if possible

```
GA-3: Sort salaries using Selection sort and Bubble sort (simple version)

n = int(input("Enter number of employees: "))
salaries = []
for i in range(n):
salaries.append(float(input("Enter salary: ")))



def selection_sort(arr):
for i in range(len(arr) - 1):
min_index = i
for j in range(i + 1, len(arr)):
if arr[j] < arr[min_index]:
min_index = j
arr[i], arr[min_index] = arr[min_index], arr[i]
return arr



def bubble_sort(arr):
for i in range(len(arr) - 1):
for j in range(len(arr) - 1 - i):
if arr[j] > arr[j + 1]:
arr[j], arr[j + 1] = arr[j + 1], arr[j]
return arr
```

```
print("\nSelection sort:", selection_sort(salaries.copy()))
bubble = bubble_sort(salaries.copy())
print("Bubble sort :", bubble)

print("\nTop 5 highest salaries:")
count = 0
for i in range(len(bubble) - 1, -1, -1):
if count == 5:
break
print(bubble[i])
count = count + 1
```